CAPUEE 25-26

Intro to Dashboards with Streamlit

Adriano Caprara

adriano.caprara@upc.edu

Agenda

CITCEA UPC
Boosting the future of energy

1. Dashboard intros
2. Widgets
3. Development
4. Deployment and scalability
5. Practical session

Introduction

Dashboards - intro

Streamlit
https://streamlit.io/gallery
https://cheat-sheet.streamlit.app/
https://30days.streamlit.app/?challenge=Day1
http://prettymapp.streamlit.app/

Dash by Plotly
https://plotly.com/examples/

Tableau
https://www.tableau.com/dashboard/dashboard-examples

Power BI, Google data studio…

Streamlit

- Simple syntax if already familiar with Python
- No need for HTML, CSS or JavaScript knowledge
- Integration with Python ecosystem

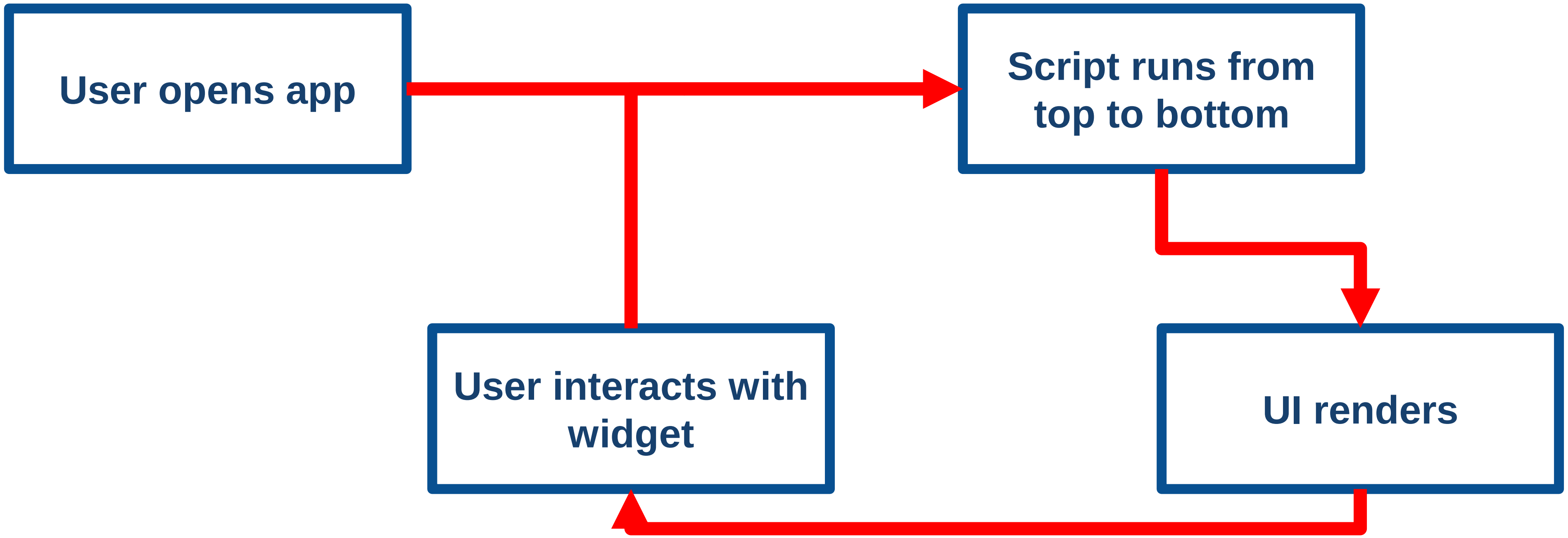
How does it work?

Code execution flow: execute the script top to bottom everytime a user interacts with a widget(i.e. dashboard.py) and Code execution flow

Automatic UI generation: commands such as «st.title(), st.dataframe(), st.plotly_chart» are already integrated with corresponding HTML and CSS to be rendered in a web browser

Deployment: with minimal configuration streamlit dashboard can be deployed on various platforms (Streamlit Cloud, Heroku, AWS,…)

Execution flow



Install Streamlit and write dashboard script

•From your terminal:

```
> pip install streamlit
```

- **st.title()** and **st.write()** to generate titles and texts without the need for css/html styling

dashboard.py first_app.py import_api_data.py plug_data.csv clean_plug_data.py electricity_price_data.csv

```python
import streamlit as st
import pandas as pd

# Load data
df_price = pd.read_csv( filepath_or_buffer: "electricity_price_data.csv", index_col=0, parse_dates=True)


st.title('My first app! This isn the st.title() function.')

st.write("Welcome to my first app! This is the st.write() function. It is used to write text in the app.")

st.dataframe(df_price.iloc[0:5])
```

- Built-in functions to work with well known python data structures – **st.dataframe(df)**

Run dashboard from the terminal

CITCEA UPC
Boosting the future of energy

In the terminal: **streamlit run your_script.py**

```
(S52_conda) PS G:\Il mio Drive\digital_energy\Code repos\visualization class> streamlit run first_app.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.1.37:8501
```

Visualize in the browser!

My first app! This isn the st.title() function.

Welcome to my first app! This is the st.write() function. It is used to write text in the app.

datetime	price	res_share
2023-07-06 00:00:00+02:00	163.57	0.5668
2023-07-06 01:00:00+02:00	158.54	0.5688
2023-07-06 02:00:00+02:00	146.27	0.5708
2023-07-06 03:00:00+02:00	145.09	0.571
2023-07-06 04:00:00+02:00	145.51	0.5717

Widgets and functionalities

Widgets overview – st.ecc...

DISPLAY WIDGETS

(app to user)

- .write()
- .dataframe()
- .table()
- Metric()
- .plotly_chart()
- -line_chart()
- …

INPUT WIDGETS

(user to app, requires python post-processing of the interactions)

- .text_input()
- .number_input()
- .slider()
- .checkbox()
- .button()
- .date_input()
- .file_uploader()
- .multiselect()

LAYOUT WIDGETS

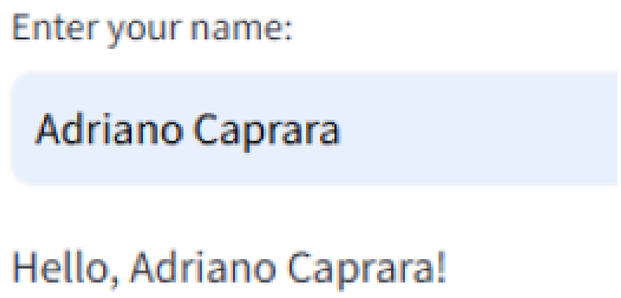
(visuals and organization)

- Col1, col2 = st.columns(2)
- Tab1, tab2 = st.tabs(2)
- st.expander()
- st.container()

Widgets basic example

Text input:

```python
# Text input widget
name = st.text_input('Enter your name:', '')

# Display the name
if name:
    st.write(f'Hello, {name}!')
```



We store the value of the dashboard’s text input in the variable “**name**” and then we can visualize it, or use it for any other type of computations in Pyhton!

First practice with streamlit

Practical tutorial

- Install streamlit
- Run dashboard with st.title() and st.write()
- Interactive widgets:

✓ Render the name of the user using st.text_input()

✓ Ask the user his age with st.slider()

✓ Render a dataframe with st.dataframe() given a checkbox condition with st.checkbox()

Visualization

- Native plots
- Plotly (interactive)

```python
import plotly.express as px
fig = px.line(df, x='date', y='price')
st.plotly_chart(fig)
```

- Matplotlib

```python
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot(x, y)
st.pyplot(fig)
```

Tabs and columns

- Tabs

```
tab1, tab2, tab3 = st.tabs(["Overview", "Details", "Settings"])
with tab1:
    st.write("Overview content")
```

- Colums

```
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Temperature", "25°C", "1.2°C")
with col2:
    st.metric("Pressure", "101 kPa", "-0.5 kPa")
```

@st.cache_data

Every time a user interacts with ANY widget... Streamlit reruns your ENTIRE script from top to bottom!

```
@st.cache_data
def expensive_function():
    # This runs only ONCE (or when inputs change)
    # Result is cached and reused!
    return result
```

- Large computations
- Loading heavy data
- API calls

Dashboard deployment

Dashboard deployment – why?

CITCEA UPC
Boosting the future of energy

- Anyone should be able to access your product
- Available 24/7 just by clicking the link
- Tracked version history and easier collaboration.
- Instead of using your laptop, your app is running somewhere in a datacenter, on 24/7

```
Local Dashboard:            Deployed Dashboard:
├─ 1 user (you)             ├─ Unlimited users
├─ 1 device (your laptop)   ├─ Any device
├─ During work hours        ├─ 24/7 availability
├─ Same location            ├─ Global access
└─ Temporary                └─ Permanent
```

The modern development workflow

CI/CD (continuous integration/continuous development)

CI:

- Developers frequently merge code into a shared repository.
- Automated tests run on every change to detect issues early.
- Ensures the codebase stays stable and integration problems are caught fast.

CD:

- Fully automates release to production—no human approval.
- Every code change that passes automated tests is deployed automatically.
- Enables rapid, reliable delivery to users.

Cloud deployment process with Streamlit

Dashboard repository structure

```
my-dashboard/
├── app.py
└── requirements.txt
```

requirements.txt is a text file with your dependencies

```
requirements.txt
matplotlib==3.10.7
numpy==2.3.5
pandas==2.3.3
plotly==6.3.0
seaborn==0.13.2
streamlit==1.51.0
```

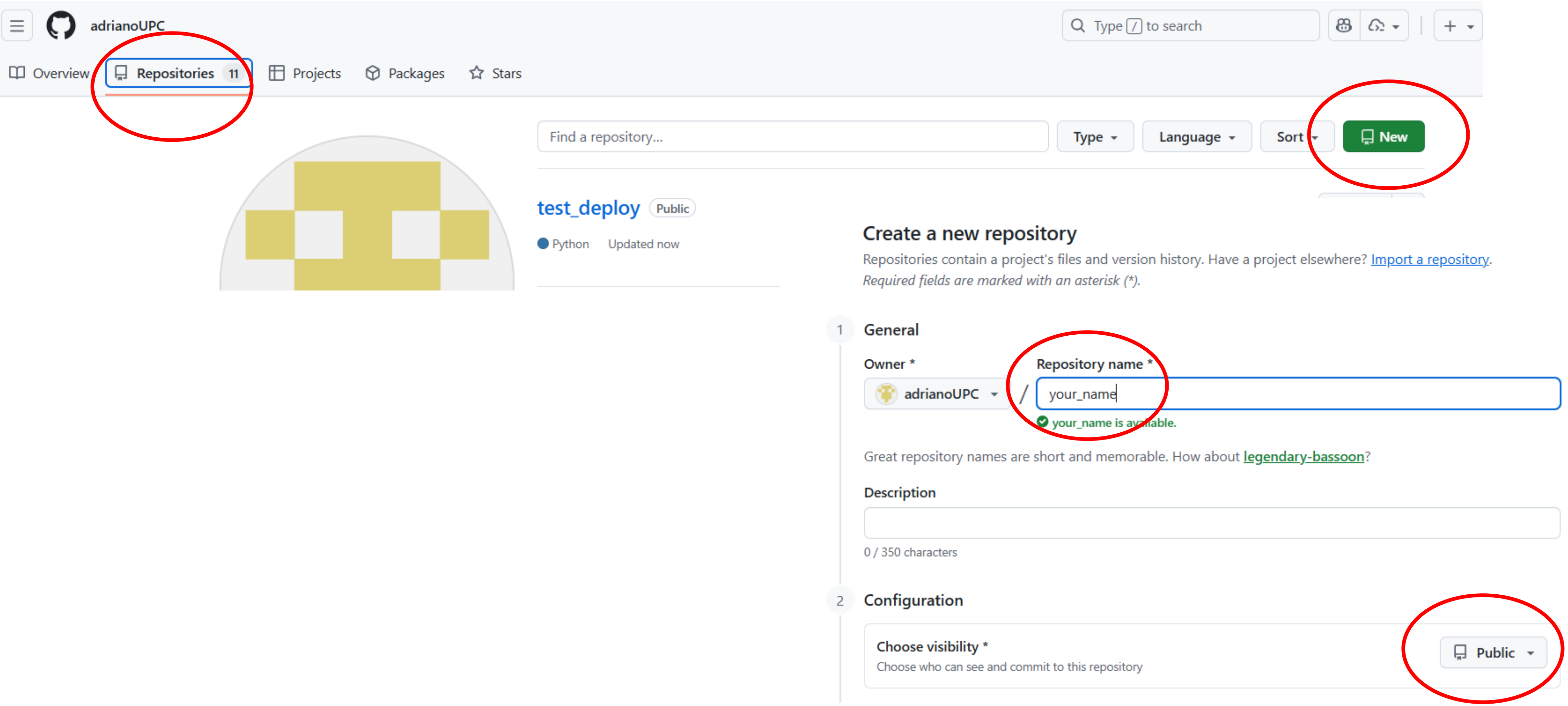
Push to github

- Create repo (**must be public)** and push code. Verify that the files appear on github
- Need to have git installed locally
- Need to have a github account (5 minutes max to create one)

Deploy on streamlit

- Once the guthub repo, the dashboard.py, and requirements are setup, just follow the steps on streamlit.io
- Very intuitive framework

Creating a github repo



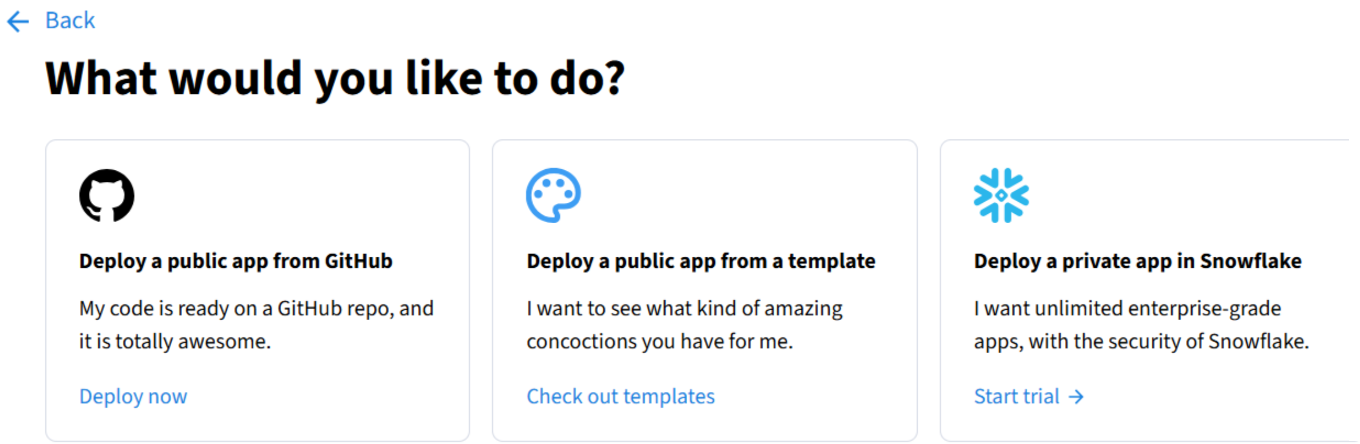
Pushing to GitHub

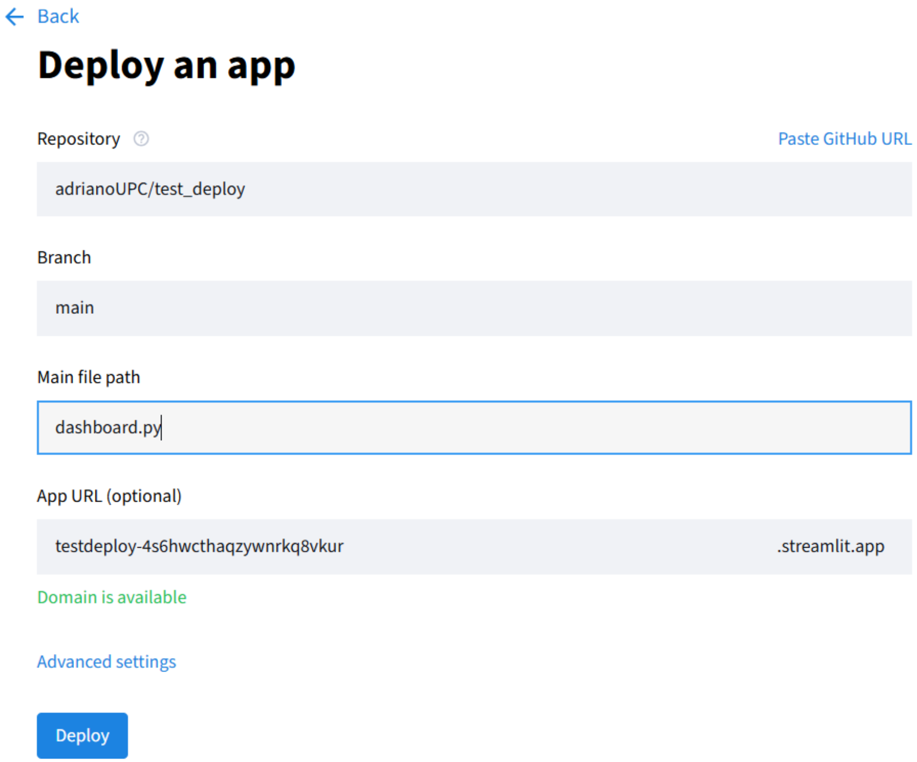
Need to have git installed locally and github account. From your terminal:

```
# Navigate to your project folder
cd path/to/my-dashboard

# Initialize git (if not already done)
git init

# Add all files
git add .

# Commit with a message

git commit -m "Initial commit - ready to deploy"

# Connect to GitHub (copy from GitHub page)
git remote add origin https://github.com/YOUR-USERNAME/my-dashboard.git

# Push to GitHub
git branch -M main
git push -u origin main
```

Streamlit.io

- We need to connect our github repo, so sign in with github and authorize streamlit
- Fill in with your repo information
- Wait a few minutes as the dashboard builds





- **Congrats! You are a data scientist!**

Usage of GenAI tools

CITCEA UPC
Boosting the future of energy

“I’m not afraid of artificial intelligence. I’m afraid of human stupidity which is natural.”

- *Cosimo Fini*

THANK YOU
FOR YOUR ATTENTION

CITCEA UPC
Boosting the future of energy

CIT UPC
TECHNOLOGY CENTER
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

tecnio catalonia
ACCIÓ
Generalitat de Catalunya